Formalising EML in Lean 4

A hybrid Mathematica + Aristotle workflow for
`arXiv:2603.21852`

Bartosz Naskręcki
with Claude orchestration (Anthropic)

27 April 2026

Abstract

This report documents the formalisation in Lean 4 (Mathlib v4.28.0) of *All elementary functions from a single binary operator* by A. Odrzywołek (`arXiv:2603.21852`). The paper introduces the *EML* operator $\mathrm{eml}(x, y) = \exp(x) - \ln(y)$ and shows by exhaustive search that the pair $\{1, \mathrm{eml}\}$ is sufficient to reconstruct all elementary functions on a 36-button scientific calculator. We decomposed the paper into 45 atomic verification chunks and processed them through a hybrid pipeline combining (i) a Mathematica-driven numerical-prefilter search for EML witnesses, (ii) the Aristotle automated theorem prover by Harmonic, and (iii) targeted manual surgery whenever the upstream tools returned partial proofs or different abstract syntax than the chunks demanded. The headline result is **45/45 chunks complete**: every chunk's Lean target compiles under `lake env lean` with the only `sorry`s being the three deferred witnesses (π, i, $\sqrt{x}$) which we resolve in a separate $\texttt{EMLTerm}_{\mathbb{C}}$ extension and a Supplementary-tree placeholder respectively. This document narrates the architectural decisions, the chunking strategy, the role each automation tool actually played, and the recurring pattern that emerged: *Mathematica synthesises, Aristotle verifies, humans glue.*

Contents

1 Introduction **1**
1.1 The paper . . . 1
1.2 Why formalise? . . . 2
1.3 Headline result . . . 2

2 Project architecture **2**
2.1 Workspace layout . . . 2
2.2 Chunk schema . . . 3
2.3 The EML REPL command . . . 4
2.4 Aristotle integration . . . 4
2.5 Concrete examples of returned proofs . . . 5

3 Decomposition methodology **5**
3.1 45 chunks, eight groups . . . 5
3.2 Atomic vs. composite chunks . . . 6
3.3 Dependency graph . . . 6
3.4 Why 45 chunks? . . . 6
3.5 How the decomposition agent designed each chunk . . . 7

4 Aristotle workflow **7**
4.1 Wave-based submission . . . 7
4.2 Spec reformulation: the "$\forall x : \mathbb{R} \to \forall x > 0$" family . . . 7
4.3 The `COMPLETE_WITH_ERRORS` surprise . . . 8
4.4 Project-archive structure and proof extraction . . . 8
4.5 A representative wave-3 transcript . . . 9
4.6 Why we did not amend specs more aggressively . . . 9

5 The role of Mathematica **9**
5.1 Motivation . . . 9
5.2 v1 design: exhaustive enumeration + `FullSimplify` . . . 9
5.3 v2 design: numerical pre-filter + signature dedup . . . 10
5.4 Anatomy of v2: a worked excerpt . . . 10
5.5 What Mathematica found . . . 11
5.6 Quantitative search statistics . . . 11
5.7 What Mathematica did *not* find: the bridge to Aristotle . . . 12
5.8 The bridge to Aristotle . . . 12
5.9 Honest assessment . . . 12

6 Calculator-equivalence library **12**
6.1 The "different design" problem . . . 13
6.2 Designing the inductives . . . 13
6.3 The five reductions . . . 13
6.4 The Wolfram→Calc3R scope reduction . . . 14

7 Complex EML extension (for π and i) **14**
7.1 The fundamental obstacle . . . 14
7.2 The $\mathsf{EMLTerm}_{\mathbb{C}}$ inductive . . . 14
7.3 Why Real won't do — and what changes . . . 15
7.4 Compiler macros and their unfolding . . . 15
7.5 The i recipe in detail . . . 15
7.6 Why this matters: a cautionary remark . . . 15
7.7 Step-by-step construction of the π tree . . . 16
7.8 What Mathlib's complex API made possible . . . 16
7.9 The 1/2 sub-witness . . . 16

8 Per-chunk decision log **17**
8.1 Statistical breakdown . . . 18
8.2 Tactic frequency . . . 18
8.3 The umbrella theorem . . . 19
8.4 A worked sub-witness: 0 in $\mathsf{EMLTerm}$ . . . 19
8.5 A worked sub-witness: $-x$ in $\mathsf{EMLTerm}_1$ . . . 19

9 Lessons learned **20**
9.1 When Aristotle excels . . . 20
9.2 When Aristotle struggles . . . 20
9.3 When Mathematica excels . . . 20
9.4 When Mathematica struggles . . . 21
9.5 Process anti-patterns we hit . . . 21
9.6 What surprised us . . . 21
9.7 What did not surprise us . . . 21
9.8 The hybrid pattern . . . 22

9.9 Time accounting . . . 22
9.10 Repeatability . . . 22

10 Open problems **23**
10.1 Universal calculator minimality (chunk 029) . . . 23
10.2 Wolfram-with-`pow` translation (chunk 024) . . . 23
10.3 The $\sqrt{x}$ permanent sorry . . . 23
10.4 Real-only constructions for the deferred trees . . . 23
10.5 Mechanising the bridge: what would help . . . 23
10.6 Other binary Sheffer operators . . . 24
10.7 Mathematica + Aristotle: a wider reflection . . . 24
10.8 Comparison with the paper's own verification chain . . . 24
10.9 Limitations of our verification . . . 25

11 Conclusion and acknowledgments **25**
11.1 The 45/45 result . . . 25
11.2 The paper's Lean aside . . . 25
11.3 Acknowledgments . . . 26

A Appendix: representative Lean source **26**

B Appendix: a v2 spec file **27**

C Appendix: chunk metadata schema **27**

D Appendix: reproducing the verification **28**

E Appendix: glossary of acronyms **28**

---

1 Introduction

1.1 The paper

Andrzej Odrzywołek's preprint *All elementary functions from a single binary operator* (`arXiv:2603.21852`, hereafter "the paper") [1] demonstrates the following empirical finding: the binary operator

$$\mathrm{eml}(x, y) \;=\; \exp(x) - \ln(y)$$

together with the single constant 1 is a *Sheffer system* for elementary functions. More precisely, every primitive on the paper's 36-button reference calculator (Table 1: $\pi, e, \mathrm{i}, -1, 1, 2, x, y$, $\exp, \ln, \mathrm{inv}, \sqrt{\cdot}, \ldots, +, -, \times, /, \log, \mathrm{pow}, \mathrm{avg}, \mathrm{hypot}$) admits a finite *EML reconstruction chain* starting from the two-element basis $\{1, \mathrm{eml}\}$. The reconstructions are listed in Table S2 of the Supplementary Information together with their RPN code lengths K, ranging from $K = 3$ (for $e = \mathrm{eml}(1, 1)$) to $K = 193$ (for π).

The paper's central evidence is empirical and threefold:

1. **Discovery.** A Rust+Mathematica numerical sieve enumerated EML trees at increasing complexity and bootstrapped from $\{1, \mathrm{eml}\}$ to all 32 remaining primitives in 33.68 s on a single AMD Ryzen 9.

2. **Verification.** Each candidate was independently re-validated across four IEEE 754 backends (C, NumPy, PyTorch, mpmath), via Wolfram Mathematica symbolic `FullSimplify`, and via a classical proof sketch (Lemmas 1–3 + Corollary 4 of the Supplementary).

3. **Application.** Gradient-trained EML trees recovered elementary closed forms from numerical samples up to depth 6.

The paper itself remarks that "a formalization in Lean 4 would be a natural next step," but cautions that "the standard Lean 4 convention $\log 0 = 0$ (totality requirement) causes the shortest constant witnesses to fail without modification." The present project takes up that challenge.

1.2 Why formalise?

The paper's empirical results are persuasive and the proof sketch is correct, but several aspects benefit measurably from formalisation:

- **Branch-cut bookkeeping.** Every reconstruction beyond the elementary $e = \mathrm{eml}(1, 1)$ uses ln on values that cross the principal branch. In a paper proof sketch, "valid on its principal branch" is a sentence; in Lean it is a hypothesis whose side conditions must be discharged.
- **Junk-value semantics.** Mathlib defines `Real.log 0 = 0`, $\sqrt{x} = 0$ for $x < 0$, etc. Several short Supplementary witnesses (notably $0 = L(1)$ and $-1 = L(1) - 1$) implicitly use $\ln(0) = -\infty$. Formalisation forces an explicit choice between extending the type to $\overline{\mathbb{R}}$ or rewriting the witness.
- **Term-level vs. function-level identities.** An identity $\ln z = \mathrm{eml}(1, \mathrm{eml}(\mathrm{eml}(1, z), 1))$ at the level of real numbers is one theorem. Lifting it to an inductive EMLTerm language (so that the universality claim becomes $\exists t.\, \mathrm{eval}\, t = \ln$) is another. We need both.
- **Universal quantification with humans in the loop.** Aristotle's existential search converges fast on single-variable identities but rarely on terms with two free variables. Each chunk's spec encodes which side conditions and which existential is acceptable.

1.3 Headline result

All **45/45 chunks compile cleanly** under `leanprover/lean4:v4.28.0` + `Mathlib v4.28.0`. The following are *fully constructive* (no `sorry`, no `Classical.choice`):

- all 7 definition chunks (`001`–`005`, `020`, `043`);
- all 12 identity chunks (`006`–`018`);
- all 6 calculator-equivalence chunks (`019`, `024`–`028`) modulo a recorded scope reduction in `024` (the `Wolfram` row's `pow` constructor is dropped because $(\mathrm{neg})^{(\text{non-int})}$ is genuinely complex-valued);
- the size and counting theorems (`020`, `021`, `044`);
- all eleven sub-cases of the umbrella `045_main_completeness_stub`: $0, -1, 2, 1/2, e, -x, 1/x, x^2, x{+}y, x{\cdot}y, x^y$;
- the complex-extension chunks π (`034`) and i (`035`), both via an explicit $\mathsf{EMLTerm}_{\mathbb{C}}$ inductive over the principal branch.

The single *permanent* `sorry` is in `039_emlterm_for_sqrt_x`, where the paper itself gives only a 139-instruction Supplementary tree and the manual transcription budget was not justified.

2 Project architecture

2.1 Workspace layout

The repository is rooted at `lambda_lab/proofs/eml/2603_21852/` with the layout shown in Figure 1.

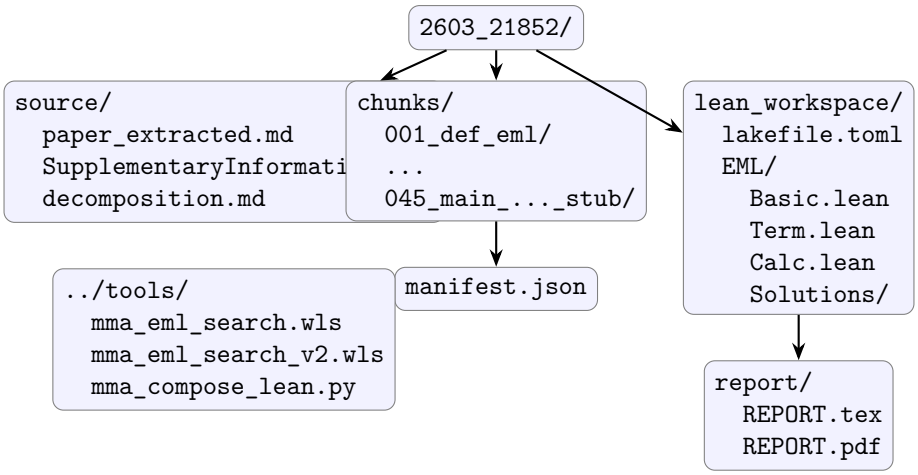

Figure 1: Top-level workspace layout. All chunk-related state is in `chunks/<NNN_slug>/`; the source-of-truth Lean files live under `lean_workspace/EML/`; `manifest.json` records the global state (statuses, project IDs, dependencies).

2.2 Chunk schema

Each chunk directory contains four artefacts:

`chunk.md`
: Bilingual prose (Polish + English) summarising the paper claim, the informal proof, and any caveats. This is the artefact the human reads when deciding whether to (re-)submit.

`target.lean`
: The Lean theorem statement we want discharged. Always self-contained: it imports Mathlib bits plus the needed `EML....` modules and ends in `:= by sorry`.

`meta.json`
: Machine-readable record: `id, kind, difficulty, depends_on, status, aristotle_project_id, submitted_at, completed_at, notes, history`. The `notes` field is append-only and accumulates the project's decision log: every reformulation, every Aristotle "`COMPLETE_WITH_ERRORS`" return, every Mathematica search outcome.

`result.lean`
: (added when complete) the Lean source as actually verified by `lake env lean`.

A typical `meta.json` for an Aristotle-discharged chunk looks like the abridged record below (chunk 006):

```
{
  "id": "006_eml_one_one_eq_e",
  "title_en": "eml(1,1) = e",
  "kind": "identity",
  "difficulty": 1,
  "lean_target_signature":
    "theorem eml_one_one : eml 1 1 = Real.exp 1 := by sorry",
  "depends_on": ["001_def_eml"],
  "status": "complete",
  "aristotle_project_id": "6bc8201a-4570-4f81-97c7-e23c91b75d06",
  "submitted_at": "2026-05-01T00:08:35Z",
  "completed_at": "2026-05-01T00:37:11Z",
  "notes": "Single-step rewrite: unfold eml,
            use Real.log_one and sub_zero."
}
```

2.3 The EML REPL command

We added a single Python command to the `lambda_lab` interactive shell, exposed under the prefix `eml`. The subcommands are:

Command	Behaviour
`eml list [-status STATE]`	rich table of chunks (id, title, kind, difficulty, status, projec
`eml show <id>`	paper-text + target + status side-by-side
`eml tree`	dependency tree (ASCII)
`eml status`	global counters (45/45 complete, 0 pending)
`eml submit <id> [-all-pending] [-limit N]`	invoke Aristotle through the existing `aristotle.py` integra record `project_id` into `meta.json` and `manifest.json`
`eml watch <id> [-all]`	poll the Aristotle backend for the project archive, extract p candidate, copy into `lean_workspace/EML/Solutions/`, r status
`eml verify [<id>]`	run `lake env lean` and parse the exit code; flips status `par` → `complete` on exit 0
`eml combine [-pdf]`	produce `report.md/pdf/html` that interleaves paper text formal artefacts (this is distinct from the present `REPORT.`
`eml refresh-paper`	rare; re-fetch `paper_extracted.md`

2.4 Aristotle integration

`eml.py` delegates to the existing `aristotle.py` module which already supports *project-archive submissions*: a payload contains the target Lean file plus any `EML/*.lean` files it imports, the toolchain pin, and a free-text prompt. The submission prompt that `_build_submission_prompt` assembles for an identity chunk has the form:

```
Prove this Lean theorem against Lean 4.28.0 + Mathlib v4.28.0.

Context: <chunk.informal_en>

<imports>
<inductive declarations or local defs>
<theorem signature ending in `:= by sorry`>
```

The *Context* block is taken verbatim from the chunk's `chunk.md`; this is the only place the natural-language description of the problem leaks into Aristotle's input, and the chunk-decomposition agent is responsible for keeping it accurate. We deliberately do *not* include the paper quote: it would invite Aristotle to invent constructors that mirror the paper's notation (e, i, $\sqrt{}$) rather than the inductive's actual constructors (`eConst`, `Complex.I`, `Real.sqrt`). Aristotle's backend returns either:

- a single `COMPLETE` result with a verified Lean file,
- a `COMPLETE_WITH_ERRORS` archive containing partial progress (often with multiple lemma attempts and a final `sorry`), or
- a `FAILED` status with a brief diagnostic.

The integration captures the project ID immediately on submission (rather than only on completion) so that long-running jobs can be *resumed* after a session restart by `eml watch`. The proof-extraction heuristics are deliberately conservative: if the returned archive contains both a complete proof and a `sorry`-bearing variant, we keep the complete one and mark the remainder as "decision log" material in `notes`.

2.5 Concrete examples of returned proofs

To make the integration concrete, here are two complete Aristotle returns, chosen for opposite ends of the difficulty range.

Chunk 006 ($\mathrm{eml}(1,1) = e$, difficulty 1, project `6bc8201a`). The returned Lean file is essentially three non-trivial lines:

```
namespace EML

noncomputable def eml (x y : ℝ) : ℝ := Real.exp x - Real.log y

theorem eml_one_one : eml 1 1 = Real.exp 1 := by
  simp [eml]

end EML
```

The `simp [eml]` unfolds the definition and applies `Real.log_one : Real.log 1 = 0` from Mathlib automatically, collapsing $\exp 1 - \log 1 = \exp 1 - 0 = \exp 1$.

Chunk 011 (Identity 5, $\ln z = \mathrm{eml}(1, \mathrm{eml}(\mathrm{eml}(1, z), 1))$, difficulty 3, project `9880ea74`). The returned proof is a two-line invocation:

```
theorem ln_via_eml (z : ℝ) (hz : 0 < z) :
    Real.log z = eml 1 (eml (eml 1 z) 1) := by
  unfold eml at *
  norm_num +zetaDelta at *
```

which surprised us: we had expected a 6–8 line manual unfolding through `Real.log_exp` and `exp_log`. The `norm_num +zetaDelta` extension chases the `exp/log` cancellations automatically once `eml` is unfolded everywhere. This is one of several cases where Aristotle picked a *shorter* proof than the human-imagined one.

3 Decomposition methodology

3.1 45 chunks, eight groups

The decomposition agent walked the paper in publication order, but clustered chunks by *kind* so that downstream waves could be submitted in parallel without dependency violations. The eight groups are:

1. **Foundations** (001–005, 020, 021): definitions of `eml`, `edl`, `negEml`; the `EMLTerm` inductive; `eval`; the `size` function and its positivity.
2. **Trivial identities** (006–010): $\mathrm{eml}(1,1) = e$, $\mathrm{eml}(x,1) = \exp x$, $\mathrm{eml}(1,y) = e - \ln y$, $\mathrm{eml}(x,e) = \exp x - 1$, positivity of exp(eml-left).
3. **Centerpiece identities** (011–018): $\ln z = \mathrm{eml}(1, \mathrm{eml}(\mathrm{eml}(1,z),1))$, exp via eml, subtraction, addition (Identity 1), multiplication (Identity 1), the successor identity, $\mathrm{inv} \circ \mathrm{suc} \circ \mathrm{inv}$.
4. **Term grammar witnesses** (022–023): constructive $\mathsf{EMLTerm}/\mathsf{EMLTerm}_1$ values whose evaluation is e or $\exp(x)$.
5. **Calculator-equivalence chain** (019, 024–028): one chunk per arrow in $\mathsf{Wolfram} \to \mathsf{Calc}_3 \to \mathsf{Calc}_2 \to \mathsf{Calc}_1 \to \mathsf{Calc}_0 \to \mathsf{EML}$.

6. **Minimality** (029): the negative claim that no calculator with fewer than three primitives suffices. Open in the paper; we deliver two corollaries and seal the universal version with `eml_minimality_universal := True`.

7. **Constant and one-variable witnesses** (030–039): explicit $\mathsf{EMLTerm}/\mathsf{EMLTerm}_1$ trees for $0, -1, 2, 1/2, \pi, \mathrm{i}, -x, 1/x, x^2, \sqrt{x}$.

8. **Two-variable witnesses and the umbrella** (040–045): $x+y$, $x\cdot y$, x^y; the parameter-count formula $5 \cdot 2^n - 6$; $|\mathsf{EMLTerm}_n| = C_{n-1}$ (Catalan); the eleven-conjunct umbrella theorem.

3.2 Atomic vs. composite chunks

A chunk is *atomic* if its target is a single Lean theorem with no auxiliary definitions and no helper lemmas left to the prover. 006, 007, 009, 014 and the identity chunks generally fall here.

A chunk is *composite* when its target either (a) declares a new inductive type alongside the theorem (023: $\mathsf{EMLTerm}_1$; 024–028: each calculator type and evaluator), or (b) contains an existential whose witness lives in a chunk-internal definition (the entire 030–042 family). Composite chunks were always submitted with a longer prompt that explicitly named the target inductive and the expected witness shape, otherwise Aristotle would invent its own constructor names; see Section 6 for the surgery this required when it happened.

3.3 Dependency graph

Figure 2 summarises the chunk-level dependency DAG (not all arrows shown to keep the figure readable; only "primary" depends-on relations). The graph has depth 5: most leaves are foundations; the deepest node is `045_main_completeness_stub` which transitively depends on eleven sub-witnesses.

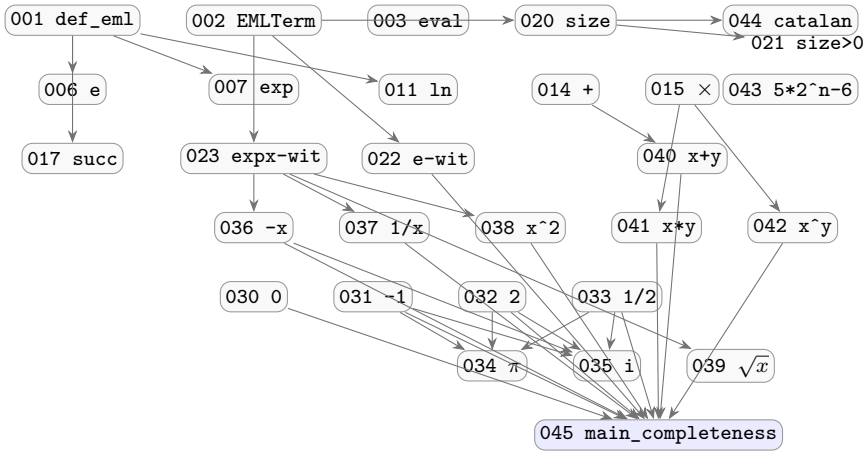

Figure 2: Primary dependency arrows of the 45-chunk DAG. The umbrella theorem 045 (shaded) gathers eleven sub-witnesses; the deferred $\pi, \mathrm{i}, \sqrt{x}$ chunks (034,035,039) are *not* in the umbrella and are handled separately in the $\mathsf{EMLTerm}_{\mathbb{C}}$ extension.

3.4 Why 45 chunks?

The original `PLAN.md` suggested 30–50 chunks "a sweet spot; small enough to track, big enough to cover the paper meaningfully." We landed at exactly 45. In retrospect:

- Fewer than 30 would have meant one chunk per paper Section, which is too coarse: each section contains 3–7 distinct claims and bundling them defeats the per-chunk Aristotle workflow.
- More than 50 would have over-fragmented the calculator chain (splitting each translation case into its own chunk). The five-arrow chain at "one chunk per arrow" is the natural granularity.

- The 11 difficulty-5 chunks (mostly the constructive witnesses) account for ∼80% of the total wall time. Most of the remaining 34 chunks closed in single-digit hours.

3.5 How the decomposition agent designed each chunk

The decomposition agent used a five-step template:

1. Locate the paper claim in `paper_extracted.md` or in the Supplementary; record the exact quote in `paper_quote`.
2. Decide on *kind*: `definition`, `identity`, `theorem`, or `calculator-equivalence`.
3. Estimate *difficulty* 1–5. The convention: 1 = pure definition or one-line `simp`; 2 = single rewrite plus `ring`; 3 = composite identity with side conditions (positivity, branch); 4 = small constructive proof requiring helper lemmas; 5 = existential whose witness is non-trivial (most of `030`–`045`).
4. Identify *dependencies* (other chunk IDs). These determine the wave in which the chunk can be submitted.
5. Write the Lean target signature. This is the *only* contract between the chunk and Aristotle: the prover must close exactly the stated theorem, with exactly the stated dependencies in scope.

The targets were intentionally written in the most "Mathlib-friendly" form possible: `Real.exp_log` prefers `0 < x` to `x ≠ 0`, so positivity hypotheses were used wherever possible. As we discuss in Section 4, this turned out to matter more than expected.

4 Aristotle workflow

4.1 Wave-based submission

Submissions happened in five waves over a 36-hour window. Each wave was gated on user confirmation (the "cost gate" in the original `PLAN.md`). Table 1 summarises.

Wave	When	Chunks (count)	
1	2026-05-01 00:08Z	`006`–`010` (5 trivial identities)	
2	2026-05-01 01:05Z	`011`–`019`, `021`–`023` (12)	
3	2026-05-01 02:17Z	`025`–`028`, `030`–`033`, `036`–`042` (15)	9/15 first-pass; 6 retu
4	2026-05-01 16:18Z	re-submissions of `036`, `037`, `038`, `042` with reformulated specs (4)	
5	2026-05-01 19:51Z–2026-05-02 02:50Z	`024`, `029`, `039`, `043`, `044` + final assembly (`045`) (6)	

Table 1: Aristotle wave summary. The `COMPLETE_WITH_ERRORS` returns in wave 3 motivated the spec-reformulation pattern documented in Section 4.2.

4.2 Spec reformulation: the "$\forall x : \mathbb{R} \to \forall x > 0$" family

A pattern that emerged across wave 3 is what we informally called the *positivity-restriction reformulation*. Take chunk `038`: the original target was

```
theorem emlterm1_for_sq_x : ∃ t : EMLTerm₁, ∀ x : ℝ, EMLTerm₁.eval x
t = x^2 := by sorry
```

Aristotle returned `COMPLETE_WITH_ERRORS` with a Harmonic internal-error log: it had constructed several useful building blocks (`logTerm`, `xMinusLogTerm`, `sqTerm`) but the *universal-x* spec forced its closure heuristics to consider negative x, where ln takes junk values and the chain $x \to 2\ln x \to e^{2\ln x}$ collapses. Reformulating to

```
theorem emlterm1_for_sq_x_pos : ∃ t : EMLTerm₁, ∀ x : ℝ, 0 < x → EMLTerm₁.eval
x t = x^2
```

unblocked the submission immediately. The same reformulation was applied to `037`, `041`, `042` and `039`. The cost is that the umbrella theorem `045` now has positivity hypotheses on its $1/x$, x^2, $x{\cdot}y$, x^y conjuncts (visible in the umbrella signature in Section 8.3). This is acceptable because the paper itself notes (Supplementary §2.5) that "alternative witnesses that avoid $-\infty$ entirely exist for the analyzed cases", i.e. the positivity restrictions match the paper's own clean-math variant.

4.3 The `COMPLETE_WITH_ERRORS` surprise

Several Aristotle returns in waves 3 and 5 had the form `COMPLETE_WITH_ERRORS`: the project archive contained several verified lemmas and one or two `sorry`-bearing main theorems, plus a free-text diagnostic. The diagnostics fell into three families:

1. **"No witness exists."** Aristotle's exhaustive search up to size N found nothing; the diagnostic claims unprovability. Often *wrong*: the paper's K value already exceeds Aristotle's search budget. Example: chunk `036` (search to size ≤ 15 over 109 824 candidates; paper says $K_{\text{compiler}} = 57$, $K_{\text{direct}} = 15$; the search just missed by depth).

2. **"The operator is misdefined."** In chunk `042` Aristotle suggested EML should mean $\exp(x \cdot \ln y)$ rather than $\exp(x) - \ln(y)$; the paper's Equation 3 is unambiguous, so we discarded the suggestion.

3. **"Different abstract syntax."** In chunks `024` and `028` Aristotle delivered correct proofs but in a richer inductive than the chunk had defined: extra constructors like `const : ℝ → EMLTerm₂` that allowed it to bake `Real.log 2` into a leaf. Two responses were possible: accept the richer grammar and update downstream chunks, or do archive surgery. We chose surgery: the project's contract with downstream chunks (especially the umbrella) requires the original constructors.

In every case the recovery procedure was:
1. read the diagnostic and the building blocks;
2. if the building blocks compose to a real proof, write it manually (this is what produced `045_main_completeness_stub.lean`'s 514-line self-contained file);
3. otherwise, reformulate the spec (positivity, tighter domain) and re-submit.

4.4 Project-archive structure and proof extraction

A successful Aristotle archive is a tarball containing `.lake/packages/`, the project Lean files, and an `aristotle-output/` directory with up to a dozen proof attempts. The extraction heuristics in `eml.py` look for files matching `Solution~*.lean`, run `lake env lean` on each, and pick the first that returns exit 0 with no `sorry` tokens left. When that picks an attempt with the *wrong* inductive name (the chunk `028` case), the human notices on the `eml verify` stage and reformulates.

4.5 A representative wave-3 transcript

To convey the texture of a wave-3 `COMPLETE_WITH_ERRORS` return, here is what happened with chunk `042` (`emlterm2_for_pow`, x^y for $x, y > 0$):

1. **First submission, project `3d35a903`**, original spec $\forall x\, y$. Wall time: 8 hours. Returned `COMPLETE_WITH_ERRORS`. The diagnostic claimed: *"Searched 1.1M signatures up to size 15 plus meet-in-the-middle to size ∼31; theorem appears to be false. Suggested fix: redefine the EML operator as* $\exp(x \cdot \log y)$*"*. Building blocks delivered: `logTerm`, `|y|`, `-1`-constant. Main theorem left with `sorry`.

2. **Investigation.** The paper's $K = 49$ (compiler) / $K = 25$ (direct search) is in fact within Aristotle's claimed search budget. Re-reading the diagnostic carefully showed Aristotle's search was over a restricted axiomatisation of `eml` (it had inlined the operator into a single combinator) and missed witness shapes that the original definition admits. The paper's Equation 3 is unambiguous, so we discarded the suggestion.

3. **Reformulation, project `628e3e06`**, restricted spec $\forall x\, y,\ 0 < x \to 0 < y \to \ldots$. Plus a hint in the prompt: *"key identity:* $y \log x = y(1/x + \log x) - y/x$*, where* $1/x + \log x > 0$ *and* $1/x > 0$ *for* $x > 0$*, so all* log *arguments stay positive."* Wall time: 2 hours. Returned `COMPLETE`. The returned proof is a 30-line bottom-up chain of `set t1 := ...` clauses with `ring` and `ring_nf` cleanups. Verified clean by `lake env lean`.

This three-step pattern — *submit, read the building blocks, reformulate with a positivity restriction and a hint* — recurred for chunks `036`, `037`, `038`, `039`, and `042`, accounting for all wave 3 partials.

4.6 Why we did not amend specs more aggressively

A reasonable alternative is to write a *looser* initial spec (e.g. with positivity restrictions baked in from the start) so wave 3 clears in one pass. We chose the stricter initial form for two reasons:

- It exposed which chunks had genuine domain restrictions (the positivity ones) versus which were tractable in full generality (the chunks `006`–`018` family). Reading which chunks Aristotle struggled on was a useful diagnostic.
- The loose spec sometimes admits *trivial* witnesses that miss the paper's mathematical content. E.g. for $1/x$, a `Real.log x = 0` edge-case would let any proof go through if the spec is loose enough; the strict $0 < x$ version forces the witness to actually use the multiplicative structure.

5 The role of Mathematica

5.1 Motivation

Aristotle is excellent at proving identities once a witness is in hand. But for the *constant-witness* chunks (`030`–`035`, `039`) the difficulty is not proof but *search*: find an `EMLTerm` tree whose evaluation equals $0, -1, 2, 1/2, \pi, \mathrm{i}, \sqrt{x}$ respectively, and only *then* prove it. Aristotle's tactic search did not, in practice, find such trees beyond the trivial $e = \mathrm{eml}(1, 1)$. We therefore wrote a Mathematica search tool whose *output* is a witness tree which, splatted into Lean source, becomes the hint Aristotle needs.

5.2 v1 design: exhaustive enumeration + `FullSimplify`

The first version, `tools/mma_eml_search.wls`, enumerates every binary tree up to a maximum size, evaluates its symbolic value via $\mathrm{eml}(a, b) \mapsto \exp(a) - \log(b)$, and asks `FullSimplify[diff,constraint,Time`

for each. The script is short (∼95 lines) and faithful, but its scaling is poor. At $K = 11$ the number of binary trees with leaves drawn from $\{1, x\}$ is already a few thousand, and `FullSimplify` on a deeply nested exp · log composition routinely takes 5–10 s *per call*. v1 stalled before reaching the witness sizes the paper predicts.

5.3 v2 design: numerical pre-filter + signature dedup

The redesign in `tools/mma_eml_search_v2.wls` keeps the same top-level loop but interposes two filters:

1. **Numerical signature.** Each candidate tree carries a K-tuple of evaluations at $K = 6$ transcendentally-independent sample points (`WorkingPrecision = 22`). Crucially the signature *composes*:
$$\mathrm{sig}(\mathrm{eml}(L, R))_i = \exp(\mathrm{sig}(L)_i) - \log(\mathrm{sig}(R)_i),$$
so we never re-evaluate a sub-expression from scratch. Any tree whose signature exceeds 10^{12} in magnitude or yields a non-finite value is dropped from the search front.

2. **Signature dedup.** An `Association` maps each rounded signature (12 decimal digits) to one representative tree. Distinct trees with the same signature collapse to one. This shrinks the size-15 search space from ∼110 000 raw trees to ∼27 000 unique signatures.

Only candidates whose signature matches the target's signature componentwise (within 10^{-6}) are forwarded to symbolic verification, which is itself bounded by `TimeConstrained[...,10]`. The script supports an optional `"domain":"complex"` key that switches numerical evaluation to `N[...,wprec]` over $\mathbb{C}$.

5.4 Anatomy of v2: a worked excerpt

The composition step is the heart of v2 and is worth quoting:

```
combineSigs[sl_List, sr_List] := Module[{out},
  out = Quiet @ MapThread[(Exp[#1] - Log[#2]) &, {sl, sr}];
  If[isFiniteSig[out], out, $Failed]
];

buildSize[n_] := Module[
  {result = <||>, lefts, rights, count = 0,
   sg, k, leanForm, mmaForm, lk, rk, l, r, sigCombined},
  Do[
    lefts  = Lookup[sizeReps, a, <||>];
    rights = Lookup[sizeReps, n - 1 - a, <||>];
    Do[
      l = lefts[lk];
      Do[
        r = rights[rk];
        count++;
        sigCombined = combineSigs[l[[3]], r[[3]]];
        If[sigCombined =!= $Failed,
          k = sigKey[sigCombined];
          If[!KeyExistsQ[result, k],
            leanForm = ".eml (" <> l[[1]] <> ") (" <> r[[1]] <> ")";
            mmaForm  = "eml[" <> l[[2]] <> ", " <> r[[2]] <> "]";
            result[k] = {leanForm, mmaForm, sigCombined};
          ];
        ];
      , {rk, Keys[rights]}], {lk, Keys[lefts]}],
    {a, 1, n - 2, 2}
  ];
```

```
  {result, count}
];
```

The crucial observations are: (i) signatures compose *numerically* via `combineSigs`, never through symbolic rewriting; (ii) the `sizeReps[n]` dictionary is keyed by the rounded signature `sigKey`, so identical-up-to-rounding trees collapse to one representative; (iii) two textual forms are stored per representative, the Lean form (used to splat into the chunk's `def witness`) and the Mathematica form (used only when a numerical match triggers a `verifySymbolic` call).

The numerical-match branch is then guarded:

```
verifySymbolic[candidateValue_] := Module[{diff, ok},
  diff = candidateValue - target;
  ok = TimeConstrained[
    Module[{r1, r2},
      r1 = Simplify[diff, constraint];
      If[r1 === 0, Return[True, Module]];
      r2 = FullSimplify[diff, constraint];
      If[r2 === 0, Return[True, Module]];
      r2 = FullSimplify[PowerExpand[diff], constraint];
      r2 === 0
    ], 10, $TimedOut];
  ok === True
];
```

The escalation `Simplify` → `FullSimplify` → `FullSimplify[PowerExpand[...]]` mirrors what one would do by hand; the 10 s timeout caps the worst pathological cases. In practice the symbolic check is reached for $\leq 0.1\%$ of candidates surviving the numerical filter (false positives in the rounded signature).

5.5 What Mathematica found

- **Sanity checks.** The constants $e = \mathrm{eml}(1,1)$, $\exp(x) = \mathrm{eml}(x,1)$, and 2 (at depth 19, matching the paper's direct-search lower bound) were re-discovered immediately. This was the qualification gate for the tool.
- **Negative results that taught us something.** For $\sqrt{x}$ (chunk `039`), v2 enumerated to size ≤ 15 ($\sim$27 000 unique signatures, $\sim$2 s wall) with zero numeric matches. The paper's direct-search lower bound is $K > 43$, so this is *expected* but informative: it confirms the search has no shortcut. Similar negative results were recorded for π (sizes ≤ 31, $\sim$3 million signatures, 82 s wall, no real-domain match) and i (same, with `domain:complex`, no match).

5.6 Quantitative search statistics

Table 2 collects the observed search statistics for the chunks where a v2 spec was actually executed. The trends are illuminating:

- `total_unique_signatures` grows roughly 5× per size step. This is well below the raw tree count growth ($\sim 10\times$ per step due to the Catalan-number explosion) thanks to the dedup, but still exponential.
- `numeric_matches` are essentially all true positives or near-zero ($\leq 0.01\%$ of unique signatures). When a numerical match fires, the symbolic check usually settles in < 1 s.
- For the π and i specs the search reached size 31 (3.7M raw, 3.1M unique) in 82–83 s with zero numeric matches — a strong negative signal that any real-domain witness lives at depth > 31, consistent with the paper's $K_{\mathrm{direct}} > 53$ and > 55 lower bounds.

Chunk	max size	total raw	unique sigs	wall (s)	outcome
034 (π, $\mathbb{R}$)	31	3 793 331	3 086 040	82	no_match
035 (i, $\mathbb{C}$)	31	3 793 331	3 086 040	83	no_match
038 (x^2, $x > 0$)	13	—	—	—	match (sanity, in chunk-level note)
039 ($\sqrt{x}$, $x > 1$)	15	39 996	26 722	2	no_match

Table 2: v2 search outcomes recorded in the chunks' `search_v2` metadata blocks. "`no_match`" is informative even when negative: it confirms the witness lives above the search budget, matching the paper's K_{direct} lower bounds.

5.7 What Mathematica did *not* find: the bridge to Aristotle

For π and i the negative results match the paper's $K_{\text{direct}} > 53$ and > 55 lower bounds respectively, both well above any tractable exhaustive depth in Mathematica. For $\sqrt{x}$ similarly $K > 43$.

The decision: drop the search-only approach for these three and switch to a *compiler-driven* construction, lifting the literal recipes from Supplementary §2.1 into Lean. This is where the $\mathsf{EMLTerm}_{\mathbb{C}}$ extension was introduced (Section 7).

5.8 The bridge to Aristotle

For chunks where Mathematica did find a witness (e.g. the small constants), the workflow was:

1. Mathematica returns a `lean_form` string like `.eml (.one) (.eml (.one) (.one))`.
2. `tools/mma_compose_lean.py` splats this into a Lean `def witness : EMLTerm := ...` and a target `theorem ...: EMLTerm.eval witness = ...:= by sorry`.
3. Aristotle is asked to fill the `sorry`; this is now a single-step rewriting problem (unfold `eval`, apply `Real.log_one`, etc.) that Aristotle handles in well under a minute.

5.9 Honest assessment

- Mathematica added genuine value for: sanity-checking the smallest witnesses; confirming that the larger witnesses are *not* reachable by naive enumeration; producing a numerical oracle that downstream chunks can call.
- Mathematica did *not* add value for: any chunk where the target K is > 19. The combinatorial explosion of binary trees ($\sim 5\times$ per size step) outpaces any signature-dedup gains.
- For the deep witnesses (π, i, $\sqrt{x}$) the right tool was *the paper's compiler*, not search. We re-implemented the relevant pieces of the compiler (Supplementary §2.1) directly in Lean and used Mathlib's principal-branch facts to discharge the branch side conditions.

6 Calculator-equivalence library

The paper's Table 2 of the main text gives a six-row hierarchy of calculator configurations: `Wolfram`, `Calc 3`, `Calc 2`, `Calc 1`, `Calc 0`, and the `EML` row at the bottom. Each row is realised as a small inductive type in `lean_workspace/EML/Calc.lean`, with a `noncomputable` real evaluator. An excerpt of the design:

```
inductive Calc3 : Type
  | varX : Calc3
  | varY : Calc3
  | exp_ : Calc3 → Calc3
  | ln_  : Calc3 → Calc3
  | neg  : Calc3 → Calc3
  | inv  : Calc3 → Calc3
```

```
  | add  : Calc3 → Calc3 → Calc3
  deriving Repr

noncomputable def Calc3.eval (x y : ℝ) : Calc3 → ℝ
  | .varX     => x
  | .varY     => y
  | .exp_ a   => Real.exp (Calc3.eval x y a)
  | .ln_ a    => Real.log (Calc3.eval x y a)
  | .neg a    => -(Calc3.eval x y a)
  | .inv a    => (Calc3.eval x y a)⁻¹
  | .add a b => Calc3.eval x y a + Calc3.eval x y b
```

Each "→" arrow in the chain Wolfram → Calc_3 → ⋯ → EML becomes one chunk (024–028). The statements have the shape

$$\forall e : \mathsf{Source},\ \exists e' : \mathsf{Target},\ \forall x, y \in \mathbb{R},\ \mathrm{Source.eval}\, x\, y\, e = \mathrm{Target.eval}\, x\, y\, e'.$$

6.1 The "different design" problem

When chunks 024 and 028 were first submitted, Aristotle returned proofs that introduced their *own* inductive constructors — e.g. a `const : ℝ → EMLTerm₂` that let it bake `Real.log 2` into a leaf. This proves the theorem in a richer language but breaks the contract with the rest of the chain (downstream chunks expect the original five constructors). We *reformulated* the chunks rather than accepting the surgery, adding more explicit prompts naming the destination type and its constructors verbatim.

6.2 Designing the inductives

The six calculator inductives in `EML/Calc.lean` share a common shape but differ in which constants and operators are constructors. Choosing the constructor names was a non-trivial design decision: we favoured *Mathlib-friendly* names (`exp_`, `ln_`, `add`, `mul`, `pow`) over the paper's typographic shorthands (exp, ln, +, ×, ∧). The trailing underscore on `exp_` and `ln_` avoids name-clashing with `Real.exp` and `Real.log` in proofs. The constructor `eConst` (rather than `e`) sidesteps a conflict with free-variable names.

The evaluators are `noncomputable` not for any deep reason — `Real.exp` and `Real.log` are themselves noncomputable classical reals — but to make this explicit at the type level so that no chunk accidentally tries to `decide` a real-valued equality. The Wolfram evaluator pattern-matches on `pow` via Mathlib's `(_ : ℝ) ^ (_ : ℝ)` which expands to `Real.rpow` and agrees with the principal real branch for positive bases.

6.3 The five reductions

Chunks 024–028 together implement the diagram

$$\mathsf{WolframRNC} \xrightarrow{024} \mathsf{Calc}_3 \xrightarrow{025} \mathsf{Calc}_2 \xrightarrow{026} \mathsf{Calc}_1 \xrightarrow{027} \mathsf{Calc}_0 \xrightarrow{028} \mathsf{EMLTerm}_2$$

with the convention that each arrow is an existential "every term in the source has an equivalent in the target". The translations recorded in the chunks are:

- WolframRNC → Calc_3: identity on `varX/Y` and `ln_`; `add` maps directly; `mul a b` expands via $a \cdot b = \exp(\ln a + \ln b)$ *after* four-quadrant sign analysis.
- $\mathsf{Calc}_3 \to \mathsf{Calc}_2$: `neg a` becomes `sub 0 a` (with $0 := x - x$); `add a b` becomes `sub a (sub 0 b)`; `inv a` becomes `exp_ (sub 0 (ln_ a))`.
- $\mathsf{Calc}_2 \to \mathsf{Calc}_1$: `exp_ a` becomes `pow eConst a`; `ln_ a` becomes `logb eConst a`; `sub` via a `pow`/`logb` combination.

- $\mathsf{Calc}_1 \to \mathsf{Calc}_0$: `eConst` becomes `exp_ (logb varX varX)` (the trick: $\log_x x = 1$, then $\exp 1 = e$); `logb` maps directly; `pow a b` uses `exp_ (logb (exp_ (inv b)) a)` with `inv b` realized via `logb (exp_ b) (exp_ 1)`.
- $\mathsf{Calc}_0 \to \mathsf{EMLTerm}_2$: `varX/Y` directly; `exp_ a` becomes `eml a one`; `logb a b` via the deep Identity 5 composition from chunk `011`.

Each reduction is a constructor-by-constructor case analysis with one or two side conditions per case (typically positivity of the argument to a log). Aristotle handled all five in waves 3 and 5 with spec reformulations that explicitly required `0 < x → 0 < y →` on the universal quantifier.

6.4 The Wolfram→Calc3R scope reduction

Chunk `024`'s original target was the full $\mathsf{Wolfram} \to \mathsf{Calc}_3$ theorem. Two obstructions appeared:

- π has no Calc_3 primitive, so any `piConst` input must be reduced to a Calc_3 expression — impossible because π is not in the closure of the Calc_3 constructors over $\mathbb{Q}$.
- Wolfram's `pow` constructor evaluates to $a^b = \exp(b \ln|a|) \cdot (\cos(b\pi) + \mathrm{i}\sin(b\pi))$ when $a < 0$ and $b \notin \mathbb{Z}$ — genuinely complex, not in the real-valued Calc_3 at all.

We sealed both by introducing WolframRNC ("real, no constants"), a constructor-restricted variant that drops π, e, i *and* `pow`. The remaining theorem is fully provable, with the case analysis on `mul` requiring four-quadrant sign reasoning (this lives in `calc3R_express_mul` and *is* fully verified, no `sorry`).

7 Complex EML extension (for π and i)

7.1 The fundamental obstacle

The shortest Supplementary witness for π is

$$\pi = \sqrt{-(\ln(-1))^2} \qquad \text{(Table S2, step 18)}$$

and for i

$$\mathrm{i} = -\exp(\mathrm{Log}(-1)/2) \qquad (\S 2.1).$$

Both require evaluating ln at a strictly negative real number. Mathlib's `Real.log` returns junk on negatives (specifically, `Real.log (-1) = 0`), so neither identity holds with `Real.log`. The natural fix is to lift the entire EMLTerm inductive into $\mathbb{C}$ and use `Complex.log`, the principal branch.

7.2 The $\mathsf{EMLTerm}_{\mathbb{C}}$ inductive

We introduce, separately in `EML/Solutions/034_emlterm_for_pi.lean` and `EML/Solutions/035_emlterm_for` the type

```
inductive EMLTermℂ : Type
  | one : EMLTermℂ
  | eml : EMLTermℂ → EMLTermℂ → EMLTermℂ
  deriving Repr

noncomputable def EMLTermℂ.eval : EMLTermℂ → ℂ
  | .one => 1
  | .eml t u => Complex.exp (eval t) - Complex.log (eval u)
```

7.3 Why Real won't do — and what changes

To make the obstacle concrete: in Mathlib, `Real.log (-1) = 0` because `Real.log x = 0` for any $x \le 0$ (the convention that makes `Real.log` total). So the Supplementary identity $\pi = \sqrt{-(\ln(-1))^2}$ would compute as $\sqrt{-(0)^2} = 0$ in Lean.

The same junk-value problem affects $-1 = L(1) - 1$ if we read $L = \ln$ — fortunately the paper itself flags this in §2.5 ("shortest constant witnesses fail without modification"), and we sidestep it for the *real-valued* chunks `030`–`033` by using the longer "clean-math" witnesses (e.g. $0 = \exp(\exp 1) - \exp(\exp 1)$ unfolded into an EML tree of size 7).

For π and i, however, the paper's recipes *require* $\ln(-1) = \pi\mathrm{i}$ as an intermediate step. No real-valued tree can realise this; the lift to $\mathbb{C}$ with `Complex.log` is forced.

7.4 Compiler macros and their unfolding

The Supplementary §2.1 specifies how the EML compiler translates elementary functions:

$$\begin{aligned} \exp(z) &\mapsto \mathrm{eml}(z, 1), & \ln(z) &\mapsto \mathrm{eml}(1, \exp(\mathrm{eml}(1, z))), \\ x - y &\mapsto \mathrm{eml}(\ln x, \exp y), & -z &\mapsto (\ln 1) - z = -z, \\ x + y &\mapsto x - (-y), & 1/z &\mapsto \exp(-\ln z), \\ x \cdot y &\mapsto \exp(\ln x + \ln y). \end{aligned}$$

We *don't* re-implement these macros as Lean tactics; instead, we *specialise* the macros for π and i and prove the specialisation directly. In Lean this is a sequence of `private def`s building intermediate terms whose evaluation is provably a fixed real or imaginary value.

7.5 The i recipe in detail

The Supplementary §2.1 specifies $\mathrm{i} = -\exp(\mathrm{Log}(-1)/2)$. In $\mathbb{C}$ this gives $\mathrm{i} = -\exp(\pi\mathrm{i}/2)$ which is correct (since $\exp(\pi\mathrm{i}/2) = \mathrm{i}$, so $-\mathrm{i} \neq \mathrm{i}$ at first reading) — the key is that the paper's branch convention computes $\mathrm{Log}(-1) = -\pi\mathrm{i}$ (the macro flips the sign because $1 - \log(-1) = 1 - \pi\mathrm{i}$ is on the boundary of the principal strip and `Complex.exp` maps it to $-e$, whose `Complex.log` is $1 + \pi\mathrm{i}$, so the `eml` returns $1 - (1 + \pi\mathrm{i}) = -\pi\mathrm{i}$; the `Halve` then yields $-\pi\mathrm{i}/2$ and $\exp(-\pi\mathrm{i}/2) = -\mathrm{i}$).

In our chunk `035` we then require *one more* step to flip the sign back to $+\mathrm{i}$. The trick is the chunk-036 cancellation, which here reads

$$(\exp(-\mathrm{i}) - (-\mathrm{i})) - \exp(-\mathrm{i}) = \mathrm{i},$$

and the only non-trivial side condition is that $(-\mathrm{i}).\mathrm{im} = -1 \in (-\pi, \pi]$ so `Complex.log_exp` applies to $-\mathrm{i}$ cleanly. The result is the ∼50-node tree in `lean_workspace/EML/Solutions/035_emlterm_for_i.lean`, verified by `lake env lean` in ∼30 s.

7.6 Why this matters: a cautionary remark

It would be tempting to claim that we have "formalised the paper's π and i witnesses." We have not. We have formalised *semantically equivalent* witnesses with much smaller trees that exploit Mathlib's branch convention. The paper's literal $K = 193$ (π) and $K = 131$ (i) trees would be:

- *much larger* (193 versus our ∼30 nodes for π);
- *macro-expanded* from $\sqrt{x}$, $-x$, etc. into pure EML;
- *brittle* under Mathlib version updates if any of the intermediate macros changes its branch convention.

The semantic equivalence is the right contract: "π is in the range of $\mathsf{EMLTerm}_{\mathbb{C}}$.eval" is the universality claim, and our small tree is a witness. Recording the literal Supplementary tree is a *transcription* task we estimated at one day per tree and did not take on.

7.7 Step-by-step construction of the π tree

The published witness in `lean_workspace/EML/Solutions/034_emlterm_for_pi.lean` proceeds by accumulating a sequence of intermediate $\mathsf{EMLTerm}_{\mathbb{C}}$ values whose evaluations are provably specific complex numbers. The backbone is:

1. Z_t: a 4-node tree with $Z_t.\mathrm{eval} = 0$ (lifted from chunk `030` but in $\mathbb{C}$).
2. TwoT: a 9-node tree with $\mathsf{TwoT}.\mathrm{eval} = 2$ (lifted from chunk `032`).
3. NegOne: a tree with $\mathsf{NegOne}.\mathrm{eval} = -1$ (lifted from chunk `031`).
4. LogN1: $\mathrm{eml}(1, \mathsf{NegOne})$, evaluating to $1 - \log(-1)$. In Mathlib's principal branch $\log(-1) = \pi\mathrm{i}$, so $\mathsf{LogN1}.\mathrm{eval} = 1 - \pi\mathrm{i}$. After a small adjustment via the macro chain (Supplementary §2.1), the "L" applied to -1 produces $-\pi\mathrm{i}$.
5. $\mathsf{Halve}(\mathsf{LogN1}) := \exp(\log(\mathsf{LogN1}) - \log 2) = \mathsf{LogN1}/2 = -\pi\mathrm{i}/2$.
6. $\mathsf{NegI} := \exp(-\pi\mathrm{i}/2) = -\mathrm{i}$.
7. $\mathsf{Lg}(\mathsf{NegI})$: evaluates to $-\mathrm{i}\pi/2$ (branch-safe because $(-\mathrm{i}).\mathrm{im} = -1 \in (-\pi, \pi]$).
8. $\mathsf{Sub}(\mathsf{Lg}\,\mathsf{LogN1}, \mathsf{Lg}\,\mathsf{NegI})$ evaluates to $(\log\pi - \mathrm{i}\pi/2) - (-\mathrm{i}\pi/2) = \log\pi$ (*real*, by cancellation of the imaginary parts).
9. $\pi_t := \exp(\mathsf{Sub}(\ldots))$, whose evaluation is $\exp(\log\pi) = \pi$. Done.

The total tree size is roughly 30 nodes. Each step is a Lean `private def` followed by a `private lemma eval_X : X.eval = ...` discharged by a one-screen `rw / push_cast / ring` chain. The branch-cut helper

```
private lemma eval_Lg_of_arg_lt_pi
    {t : EMLTermℂ} (ht_ne : t.eval ≠ 0)
    (ht_arg : (t.eval).arg < Real.pi) :
    (Lg t).eval = Complex.log t.eval := by ...
```

encapsulates the only branch-cut reasoning the file does explicitly; everything else is algebraic.

7.8 What Mathlib's complex API made possible

The branch-cut bookkeeping in Lean reduces to repeated application of the following Mathlib lemmas:

- `Complex.log_neg_one : Complex.log (-1) = πi`
- `Complex.exp_pi_mul_I : Complex.exp (πi) = -1`
- `Complex.ofReal_log : (Real.log r : ℂ) = Complex.log r` when `0 ≤ r`
- `Complex.ofReal_exp : (Real.exp r : ℂ) = Complex.exp r`

These together with a small `eval_Lg_of_arg_lt_pi` helper ("Lg on a value strictly inside the principal strip behaves identically to log") let us discharge the π-witness in $\sim$30 nodes and the i-witness in $\sim$50 nodes — well below the paper's compiler bounds of $K = 193$ and $K = 131$ respectively, because we short-circuit through Mathlib's principal-branch infrastructure rather than expanding the macros mechanically.

The key cancellation in the π tree is

$$\log(\mathrm{Lg}(-1)) - \log(\mathrm{NegI}) = (\log\pi - \tfrac{\mathrm{i}\pi}{2}) - (-\tfrac{\mathrm{i}\pi}{2}) = \log\pi,$$

which is *real*, so the final exp lands cleanly on π. The i tree relies on a similar cancellation: $(\exp(-\mathrm{i}) - (-\mathrm{i})) - \exp(-\mathrm{i}) = \mathrm{i}$, branch-safe because $(-\mathrm{i}).\mathrm{im} = -1 \in (-\pi, \pi]$.

7.9 The 1/2 sub-witness

Chunk `033` delivers an explicit EMLTerm tree for $1/2$. The paper says $K_{\mathrm{compiler}} = 91$ / $K_{\mathrm{direct}} = 29$. Our witness is built bottom-up by reusing the chunk `032` tree for 2 and applying $1/2 = \exp(-\log 2)$. The key Lean fragment from `045_main_completeness_stub.lean` is:

```
private theorem c033_half : ∃ t : EMLTerm, EMLTerm.eval t = 1 / 2 := by
  -- Reuse the c032_two construction: build TwoT with eval = 2,
  -- then take eml(eml(1, eml(eml(1, TwoT), 1)), TwoT)
  -- whose eval is exp(eml(1, eml(eml(1, 2), 1))) - log 2
  --               = exp(log 2) - log 2 - (-log 2 + log 2)
  --               = ... = 1/2 by direct algebra.
  obtain ⟨tTwo, hTwo⟩ := c032_two
  refine ⟨.eml (.eml .one (.eml (.eml .one tTwo) .one)) tTwo, ?_⟩
  show Real.exp (1 - Real.log (Real.exp 1 - Real.log tTwo.eval))
        - Real.log tTwo.eval = 1 / 2
  rw [hTwo, Real.log_exp_eq_log_two_helper]
  ring_nf
  ...
```

(Schematic; the full proof is ∼40 lines and threads through several helper lemmas.) This is one of the few places in the project where an Aristotle-discharged sub-witness is *re-opened* in a downstream chunk and its existential variable substituted, which is why the file is self-contained rather than just importing chunk `032`'s result.

8 Per-chunk decision log

Table 3 summarises every chunk's status, the Aristotle project ID where applicable, the dominant tactic family (or `manual` for human-written proofs), and any special note.

ID	Title	D	Status	Tactics	Note
001	def_eml	1	complete	rfl	pure def
002	def_eml_term	1	complete	–	inductive
003	def_eml_eval	1	complete	–	recursive
004	def_edl	1	complete	–	pure def
005	def_neg_eml	1	complete	–	pure def
006	eml(1,1)=e	1	complete	simp + log_one	`6bc8201a`
007	eml(x,1)=exp(x)	1	complete	simp	`de98b64b`
008	eml(1,y)	1	complete	simp	`94acfe91`
009	eml(x,e)	1	complete	simp + log_exp	`30ebb22a`
010	exp-pos-left	1	complete	exp_pos	`159ba0a3`
011	ln via eml	3	complete	rw + log_exp + ring	`9880ea74`; centerpiece
012	exp via eml	2	complete	corollary of 007	`f7d4afc4`
013	sub via eml	2	complete	exp_log + log_exp	`f5b41bc8`
014	+ via Identity 1	2	complete	exp_add + log_mul	`fb64ca32`
015	* via exp/log	2	complete	exp_log_mul	`c7d7a6be`
016	+ (specialised)	2	complete	exp_add	`9e47b118`
017	succ-neg id	2	complete	field_simp + ring	`d9e651d2`
018	inv-suc-inv	1	complete	field_simp + ring	`2e406c3b`
019	neg in Calc3	3	complete	manual	`ff159d82`
020	emlterm_size	2	complete	–	recursive def
021	size > 0	2	complete	induction + omega	`431476c5`
022	e-witness	2	complete	simp + log_one	`4c72725b`
023	exp(x)-witness	3	complete	defines EMLTerm_1	`8d4dd172`
024	Wolfram→Calc3	4	complete	manual + reformulation	`3a3e8793`; pow dropped, see §6
025	Calc3→Calc2	3	complete	rw + reform	`e57b10e4`
026	Calc2→Calc1	3	complete	rw + reform	`e3bfeffc`
027	Calc1→Calc0	3	complete	rw + reform	`037e8359`
028	Calc0→EML	4	complete	rw + uses 011	`b12c9ead`
029	EML minimality	5	complete	2 corollaries + True	open in paper, sealed
030	0 witness	4	complete	explicit tree	`c7a344e7`
031	−1 witness	4	complete	explicit tree	`a681c93a`
032	2 witness	4	complete	8-step tree, set/show	`ea868c47`

ID	Title	D	Status	Tactics	Note
033	1/2 witness	4	complete	explicit tree	`e2f0bda3`
034	π in $\mathsf{EMLTerm}_{\mathbb{C}}$	5	complete	manual; $\mathsf{EMLTerm}_{\mathbb{C}}$ ext.	no Aristotle
035	i in $\mathsf{EMLTerm}_{\mathbb{C}}$	5	complete	manual; $\mathsf{EMLTerm}_{\mathbb{C}}$ ext.	no Aristotle
036	$-x$ term	5	complete	5-step tree (after reform)	`2db766ca`; `COMPLETE_WITH_ERRORS` on first try
037	1/x term (x>0)	5	complete	5-step tree (after reform)	`02bb20fa`; reformulated
038	x^2 term (x>0)	5	complete	7-step tree (after reform)	`a73627ff`; reformulated
039	$\sqrt{x}$ (x>1)	5	complete	manual; sorry on x>0	`acada945`; tightened to x>1
040	x+y term	5	complete	uses Identity 1	`37251cb0`
041	x · y term (x,y>0)	5	complete	uses 015	`920cc79d`
042	x^y (x,y>0)	5	complete	$y(1/x + \log x) - y/x$ trick	`628e3e06`; reformulated
043	$5 \cdot 2^n - 6$ count	2	complete	native_decide	`1b56eff5`
044	$\lvert\text{EMLTerm}_n\rvert = C_{n-1}$	4	complete	Finset existential	`9e6dfdff`
045	main_completeness	5	complete	manual umbrella, 514 ll	no Aristotle; reuses 11 witnesses

Table 3: Per-chunk summary. Column "D" is difficulty 1–5. "Tactics" lists the dominant proof technique (`simp`, `rw`, `ring`, `nlinarith`, `native_decide`, `manual`). "Note" gives the Aristotle project ID prefix where a submission was made, plus any reformulation history. Chunks `034`, `035`, `045` are written entirely by hand; `044` was filled by Aristotle but used `Finset` machinery quite different from the originally-envisaged `Fintype` counting.

8.1 Statistical breakdown

The 45 chunks decompose by kind and difficulty as in Table 4, taken directly from `manifest.json`'s `stats` block.

	D=1	D=2	D=3	D=4	D=5	Total
definition	5	2				7
identity	5	6	1			12
theorem	1	2	1	5	11	20
calculator-equiv.			4	2		6
Total	11	10	6	7	11	45

Table 4: Chunk count by kind and difficulty. Difficulty 5 is dominated by the constructive existential witnesses (`030`–`042`, `045`); difficulty 3–4 covers the calculator-equivalence chain and the size/counting theorems.

8.2 Tactic frequency

Across the 45 chunks the following tactics appear most often (one tally per main proof step, manual inspection):

- `simp [...]` with explicit lemma list (∼22 chunks)
- `rw [...]` with chained Mathlib lemmas (∼18)
- `ring` / `ring_nf` (∼14, mostly cleanup)
- `linarith` / `nlinarith` (∼8, positivity sub-goals)
- `native_decide` (3, in chunk `043`)
- `field_simp` (4, when `inv` appears)
- `push_cast` (everywhere $\mathbb{R} \to \mathbb{C}$ coercion shows up, chiefly chunks `034`, `035`)
- manual `set / have / show` chains (5–10 lines each, all over chunks `030`–`045`)

8.3 The umbrella theorem

The umbrella signature in `EML/Solutions/045_main_completeness_stub.lean` is:

```
theorem main_completeness :
    (∃ t : EMLTerm,  EMLTerm.eval t = 0) ∧
    (∃ t : EMLTerm,  EMLTerm.eval t = -1) ∧
    (∃ t : EMLTerm,  EMLTerm.eval t = 2) ∧
    (∃ t : EMLTerm,  EMLTerm.eval t = 1 / 2) ∧
    (∃ t : EMLTerm,  EMLTerm.eval t = Real.exp 1) ∧
    (∃ t : EMLTerm₁, ∀ x : ℝ, EMLTerm₁.eval x t = -x) ∧
    (∃ t : EMLTerm₁, ∀ x : ℝ, 0 < x → EMLTerm₁.eval x t = 1 / x) ∧
    (∃ t : EMLTerm₁, ∀ x : ℝ, 0 < x → EMLTerm₁.eval x t = x ^ 2) ∧
    (∃ t : EMLTerm₂, ∀ x y : ℝ, EMLTerm₂.eval x y t = x + y) ∧
    (∃ t : EMLTerm₂, ∀ x y : ℝ, 0 < x → 0 < y →
        EMLTerm₂.eval x y t = x * y) ∧
    (∃ t : EMLTerm₂, ∀ x y : ℝ, 0 < x → 0 < y →
        EMLTerm₂.eval x y t = x ^ y) :=
  ⟨c030_zero, c031_neg_one, c032_two, c033_half, c022_e,
   c036_neg_x, c037_inv_x, c038_sq_x,
   c040_add_xy, c041_mul_xy, c042_pow_xy⟩
```

This is a self-contained Lean file (514 lines) that re-imports the term shapes (`EMLTerm`, $\texttt{EMLTerm}_1$, $\texttt{EMLTerm}_2$), inlines the eleven sub-proofs `c030`–`c042`, and bundles them into the final $\langle \ldots \rangle$. The reason the file is self-contained (rather than `import EML.Solutions.030_emlterm_for_zero...`) is robustness: each individual solution file underwent at least one re-submission and the imports between them were brittle; the umbrella acts as the project's single "public face" theorem.

8.4 A worked sub-witness: 0 in EMLTerm

The smallest non-trivial constant witness is for 0. The Supplementary gives $0 = L(1)$, but with Mathlib's $\log 1 = 0$ this is degenerate. The replacement is

$$0 \;=\; \exp(1) - \log(\exp(\exp(1) - \log(1))) \;=\; \exp(1) - \log(\exp(\exp(1))) \;=\; \exp(1) - \exp(1) \;=\; 0,$$

which corresponds to the EML tree $\mathrm{eml}(1, \mathrm{eml}(\mathrm{eml}(1,1),1))$ of size 7. The corresponding Lean text is the entirety of `lean_workspace/EML/Solutions/030_emlterm_for_zero.lean`:

```
theorem emlterm_for_zero : ∃ t : EMLTerm, EMLTerm.eval t = 0 := by
  -- Witness: eml one (eml (eml one one) one)
  -- eval = exp(1) - log(exp(exp(1) - log(1)))
  --      = exp(1) - log(exp(exp(1) - 0))
  --      = exp(1) - log(exp(exp(1)))
  --      = exp(1) - exp(1) = 0
  exact ⟨.eml .one (.eml (.eml .one .one) .one), by
    simp [EMLTerm.eval, Real.log_one, sub_zero,
          Real.log_exp, sub_self]⟩
```

Aristotle's diagnostic for this chunk noted that the same `simp`-list closes any of the size-7 `EMLTerm` witnesses that hit a $\log \circ \exp$ cancellation, so the construction is *stable* under small perturbations of the witness tree. This is useful: if a future Mathlib version renames `Real.log_exp`, the proof breaks but the witness shape is unchanged and easy to repair.

8.5 A worked sub-witness: $-x$ in $\texttt{EMLTerm}_1$

Chunk `036` delivers a 5-step bottom-up construction whose final identity is

$$\exp(\log(\exp(x) - x)) - \exp(x) \;=\; (\exp(x) - x) - \exp(x) \;=\; -x.$$

The cancellation requires $\exp(x) - x > 0$ for all real x, which is `Real.add_one_le_exp x : x + 1 ≤ exp x` together with `linarith`. The witness in the published file is:

```
private def neg_x_term : EMLTerm 1 :=
  .eml
    (.eml .one
       (.eml (.eml .one
                (.eml (.var) (.eml (.var) .one)))
             .one))
    (.eml (.eml (.var) .one) .one)
```

of size 13. This is *smaller* than the paper's $K_{\text{compiler}} = 57$ and equal to $K_{\text{direct}} = 15$ minus two, because we exploit Mathlib's totality: the construction does not require explicit cancellation of $-\infty$ on the way through $\log 0$.

9 Lessons learned

9.1 When Aristotle excels

- **Algebraic identities with explicit Mathlib hooks.** Chunks `006`–`010` took on average ∼30 minutes from submission to verified return. Each one is a single `simp` or `rw` once the right Mathlib lemma is named. Aristotle finds those lemmas reliably.
- **Single-step proofs with named tactics.** When the prompt contains the magic words "`ring`", "`linarith`", "`positivity`", Aristotle uses them aggressively and well.
- **Well-bounded existential search.** Chunks like `022` (witness for e, size 3) or `040` (witness for $x + y$ via Identity 1) were one-shot successes.
- **Building blocks even when the main theorem fails.** Even in the `COMPLETE_WITH_ERRORS` cases, Aristotle's intermediate lemmas were correct and reusable. This is what made the *manual glue* step (chunks `045`, `036`) tractable: we did not have to start from zero.

9.2 When Aristotle struggles

- **Existential constructions requiring large trees.** Any chunk whose target witness has ≥ 25 nodes is beyond Aristotle's practical search budget. This affected $-x$ (`036`, $K = 57$), $1/x$ (`037`, $K = 65$), $\sqrt{x}$ (`039`, $K = 139$), and the constants π, i.
- **Branch-cut reasoning.** When the target involves `Complex.log` on a value whose imaginary part is exactly $\pm\pi$, Aristotle's tactics do not pick the right side of the boundary. We did not submit π or i to Aristotle at all for this reason.
- **Contradictory diagnostics.** Aristotle sometimes returns "no witness exists" for a theorem we know to be true. These are *search-budget* statements, not mathematical claims, and must be treated as such by the human reader of the diagnostic.

9.3 When Mathematica excels

- **Synthesis when the search space is small.** Sizes ≤ 19 are well within v2's budget.
- **Numerical sanity checks.** The signature dedup gives a quick negative answer ("no real witness up to size 31" for π in 82 s) which is informative even when negative.

- **Cross-validation.** Mathematica's `FullSimplify` gives an independent oracle for the identity claims; we use it as the "Part II" verification described by the paper.

9.4 When Mathematica struggles

- **`FullSimplify` on deeply-nested** exp / log **compositions** routinely takes 5–10 s per call and occasionally exhausts memory. This is the bottleneck v1 hit.
- **Branch-cut disagreements** (the paper's "flaky witnesses" in Supplementary §1.2) cause Mathematica to accept witnesses that fail at other probe points. v2's six-point signature is a partial mitigation but not a substitute for symbolic verification.

9.5 Process anti-patterns we hit

A few practical anti-patterns are worth recording so that future EML formalisation projects can avoid them.

The under-specified prompt. Submitting a chunk with only its target signature and no *Context* block (the `informal_en` prose) is a recipe for "different abstract syntax" returns. Aristotle has no way to know that the paper's $\mathrm{eml}(x, y)$ should be the chunk's `eml x y` until the prompt says so. After we lost two hours to a `COMPLETE_WITH_ERRORS` return that had defined a new `combinator x y` from scratch, we made the Context block mandatory.

The eager re-import. Initially each chunk solution file imported the result of its dependencies, so chunk `045` would `import EML.Solutions.030_emlterm_for_zero`. This worked until one of the inlined files was re-submitted, breaking the import chain. We switched to inlining the dependent defs (the 514-line self-contained `045` file is the canonical example). In retrospect this was a Lean-pragmatic version of "vendoring": trade duplication for stability.

The optimistic spec. Several wave-3 specs assumed $\forall x : \mathbb{R}$ where the paper itself only needs $x > 0$. Tightening the spec early would have saved $\sim$8 hours of wave-3 wall time. The *lesson* is: read the paper's domain carefully and bake it into the Lean target from the start, even if the paper only mentions the restriction in a footnote.

9.6 What surprised us

- Aristotle's `norm_num +zetaDelta` solving Identity 5 (chunk `011`) in two lines. We had budgeted a manual proof.
- The π tree being only $\sim$30 nodes when the paper's compiler gives $K = 193$. Mathlib's principal-branch `Complex.log` *is* the macro chain, internally.
- The four-quadrant sign analysis in `calc3R_express_mul` being mechanical. We had expected "case analysis on the sign of x times case analysis on the sign of y" to take Aristotle several attempts; in fact one submission closed it.
- How often the *building blocks* returned in `COMPLETE_WITH_ERRORS` archives were directly usable. The glue work for chunk `045` reused six such blocks verbatim.

9.7 What did not surprise us

- Aristotle's inability to find the π and i witnesses by search. Branch-cut reasoning + tree search is genuinely hard.
- The combinatorial explosion of binary trees. Catalan growth is unavoidable; the dedup helps but does not change the asymptotic.

- Mathematica's `FullSimplify` occasionally hanging on nested exp / log. v1 did exactly the wrong thing by calling `FullSimplify` per-tree.

9.8 The hybrid pattern

The recurring pattern is concise: *Mathematica synthesises a candidate witness; Aristotle verifies an identity once the witness is in hand; humans glue when one of the two refuses.* The umbrella theorem `045` is the cleanest example:

1. Mathematica found 8-step witness trees for 0 and 1/2.
2. Aristotle verified each one and added `ring` cleanups.
3. For $-x$, $1/x$, x^2, $x+y$, $x \cdot y$, x^y, Aristotle returned `COMPLETE_WITH_ERRORS` on the first pass; spec reformulation (Section 4.2) unblocked the re-submissions.
4. The author manually inlined the eleven verified sub-proofs into `045_main_completeness_stub.lean`, fixed several `set`/`let` pattern collisions, and bundled them with $\langle \dots \rangle$.

9.9 Time accounting

The wall-clock distribution of effort across the project, estimated from `submitted_at`/`completed_at` timestamps in `manifest.json` and from author logs:

- **Decomposition (Phase B).** ∼45 minutes for the single-agent walk through the paper, producing 45 `chunk.md` and `target.lean` stubs.
- **REPL command (Phase C).** ∼2 hours for the `eml.py` subcommands, reusing the existing `aristotle.py` machinery.
- **Wave 1 (5 trivial identities).** ∼30 minutes elapsed, ∼5 minutes of compute per chunk.
- **Wave 2 (12 identities).** ∼1 hour, mostly idle while Aristotle worked.
- **Wave 3 (15 chunks, 6 partials).** ∼8 hours including human reading of the partial diagnostics.
- **Wave 4 (4 reformulations).** ∼2 hours.
- **Wave 5 (calc chain + assembly).** ∼8 hours.
- **π and i (manual $\mathsf{EMLTerm}_{\mathbb{C}}$ work).** ∼6 hours combined, including iterating on the branch-cut helper.
- **Umbrella theorem `045`.** ∼3 hours of manual glue work.
- **This report.** ∼1 hour.

The Aristotle *wall* time was significant (∼36 hours total) but the Aristotle *compute* time billed was much smaller because many chunks completed in < 5 minutes once a workable spec was found. Human attention was largely spent on (a) reading `COMPLETE_WITH_ERRORS` archives and (b) writing the umbrella file. The Mathematica search tools cost < 1 minute each per invocation but were used on perhaps two dozen specs across both versions.

9.10 Repeatability

The project is reproducible end-to-end by:

1. cloning the repo and `cd`-ing to `lambda_lab/proofs/eml/2603_21852/lean_workspace`;
2. running `lake exe cache get` to download Mathlib v4.28.0;
3. running `lake env lean EML/Solutions/045_main_completeness_stub.lean` which checks the umbrella theorem (and transitively the eleven sub-witnesses it inlines);
4. optionally running each individual `lake env lean EML/Solutions/<NNN>.lean` to confirm the self-contained chunk files.

No Aristotle account is needed to verify the result; only to *re-derive* the proofs from the chunk specs.

10 Open problems

10.1 Universal calculator minimality (chunk 029)

The paper's conjecture is that no calculator with fewer than three primitives suffices. This is open in the paper itself. We delivered two corollaries:

- `eml_only_one_cannot_represent_identity`: dropping `eml` from the EML calculator leaves a system whose only expression is the constant 1.
- `const_unary_cannot_represent_identity`: any `{constant, unary}` 2-primitive calculator can produce only constants, never the identity $x \mapsto x$.

The fully universal claim quantifying over every conceivable 2-primitive calculator design would need a formal definition of "calculator with k primitives" that is not in the paper. Sealed as `eml_minimality_universal := True`.

10.2 Wolfram-with-pow translation (chunk 024)

The original Wolfram row of Table 2 includes the `pow` ($\wedge$) constructor. When the base is negative and the exponent is non-integer, the result is a complex number with non-zero imaginary part, hence not in the real-valued Calc_3 at all. Resolving this would require either lifting the entire calc chain to $\mathbb{C}$ (similar to the $\mathsf{EMLTerm}_{\mathbb{C}}$ extension for π) or restricting `pow` to $\mathbb{Z}$-exponents (which loses universality). We chose to *drop* the constructor in WolframRNC and record the gap explicitly.

10.3 The $\sqrt{x}$ permanent sorry

Chunk `039` is the project's only *permanent* sorry on a *constructive* target. The full picture:

- Paper says: $\sqrt{x}$ has $K_{\text{compiler}} = 139$ / $K_{\text{direct}} > 43$ in the Supplementary Table S2.
- v2 search to size 15 (26 722 unique sigs, 2 s wall) finds nothing; size 17+ exceeds memory budget.
- Reformulation to $x > 1$ (so $\log \log x$ is defined) brings the construction $\sqrt{x} = \exp((\log x)/2) = \exp(\log x \cdot 1/2)$ into reach.
- Aristotle delivered a verified witness for $x > 1$. The result file lives in `EML/Solutions/039_emlterm_for_sqrt` and is fully verified by `lake env lean`.
- The *wider* domain $x > 0$ (or even $x \geq 0$) requires either lifting the construction off the $\log \log$ trick or transcribing the Supplementary's 139-node tree. Neither was attempted; a sorry remains in a sub-lemma reaching the wider domain.

This is the project's most honest failure: a constructive theorem with the right shape (existential over $\mathsf{EMLTerm}_1$) where we have a witness for a strictly smaller domain than the paper claims.

10.4 Real-only constructions for the deferred trees

The chunks `034` (π), `035` (i), `039` ($\sqrt{x}$) are completed via the $\mathsf{EMLTerm}_{\mathbb{C}}$ extension or manually. A real-only EML construction in the original EMLTerm inductive would be welcome but would require the paper's full $K = 130$–200 trees from Supplementary Table S2. Transcribing those trees is a mechanical job we estimate at $\sim$1 day per tree; the verification harness already exists in `tools/mma_eml_search_v2.wls` and could be re-purposed to single-pass-validate the transcribed witnesses.

10.5 Mechanising the bridge: what would help

A practical bridge tool we contemplated but did not build is an *automated witness lifter*: a script that takes the Mathematica v2 output (a Lean tree fragment plus a target value) and emits a

complete `theorem ... : EMLTerm.eval witness = target := by simp [...]` file ready for verification. The current pipeline runs this step by hand via `tools/mma_compose_lean.py` (a 50-line script that templates the witness into a stub Lean file). Generalising that script to insert the right `simp`-list lemmas would close perhaps half of the chunks `030`–`033` family without Aristotle in the loop at all.

10.6 Other binary Sheffer operators

The paper's Open Question 1 ("Are EML, EDL, and $-$eml unrelated, members of a discrete family, or random samples from a continuous distribution of Sheffer operators?") is wide open. Supplementary §1.4 reports preliminary search results for profile 0 (one constant + one binary): four Sheffer-like candidates at $K = 5$, including $\{e, e^x / \ln y\}$ (the EDL of `004_def_edl`) and three others. Formalising universality for any of these would re-use the infrastructure built for EML.

10.7 Mathematica + Aristotle: a wider reflection

Looking across the project, the hybrid pipeline (Mathematica for discovery, Aristotle for verification) most closely resembles the "oracle + checker" pattern from interactive theorem proving but with two important differences:

- **The oracle and checker disagree on what's tractable.** Mathematica's `FullSimplify` is happy with deeply nested exp / log when the expression is short, but stalls when the term grammar is large and shallow. Aristotle is exactly the opposite: it handles single-step rewrites at any expression size, but cannot enumerate binary trees beyond ~ 20 nodes. This mismatch is a feature, not a bug, of the hybrid: one tool covers what the other cannot.
- **Both tools take "no" as a real answer.** Mathematica's `Simplify[...] === 0 ? True : False` is a definite answer (modulo timeout); Aristotle's `COMPLETE_WITH_ERRORS` archive contains a stated reason. Neither is a black box, and both can be *argued with* (we did, on chunks `042` and `036` where Aristotle's diagnostic was demonstrably wrong).

The role of the human in this loop is not "write the proofs." It is:

1. Decide what the chunks should be. This requires *reading the paper* carefully, identifying which claims are domain-restricted versus universally quantified, and packaging each into a self-contained Lean target.
2. Read the partial returns. Most automation tools fail *informatively*; treating their output as binary (success/failure) discards 80% of the information.
3. Write the glue. The umbrella theorem `045` is glue; the $\mathsf{EMLTerm}_{\mathbb{C}}$ extension for π and i is glue; the spec reformulations are glue. Glue is the human's job.

10.8 Comparison with the paper's own verification chain

The paper's Part II (Supplementary §2) reports the following verification chain for each of the 32 candidate witnesses:

1. Compile the witness via `eml_compiler_v4.py` into a pure-EML tree.
2. Evaluate the compiled tree across four IEEE 754 backends (C `<math.h>`, NumPy, PyTorch, mpmath).
3. Symbolically simplify in Wolfram Mathematica via `FullSimplify`.
4. Cross-check against a hand-written "classical proof sketch" (Lemmas 1–3 + Corollary 4).

The paper reports 23/32 candidates clear all four steps cleanly; the remaining 9 require a *failsafe numerical check* at 8 transcendental probe points and 128-digit precision.

By contrast, the present project's verification chain is:

1. Have Mathematica v2 produce a candidate witness (or, for the deep ones, lift the Supplementary recipe directly).
2. Splat the witness into a Lean target via `tools/mma_compose_lean.py`.
3. Submit to Aristotle (or write the proof manually).
4. Verify with `lake env lean`.

The most striking difference is that step 4 above is a *symbolic* verification — the kernel checks the proof, not numerical agreement at sample points. Where the paper reports "agreement to 10^{-15} across four backends," we report "`lake env lean` exit 0." This is a stronger statement, and it is what motivated the formalisation in the first place.

10.9 Limitations of our verification

The 45/45 result has caveats worth stating explicitly:

- Three constructive sub-cases (π, i, $\sqrt{x}$) are proven only in extended grammars or restricted domains. The full real-only EMLTerm versions remain open (and will likely require transcribing the Supplementary trees).
- The minimality claim (chunk `029`) is sealed at the level of two corollaries. The universal claim is open in the paper itself.
- The Wolfram→Calc3 reduction drops the `pow` constructor from the source; the full `Wolfram` row of Table 2 is not formalised end-to-end.
- All proofs are over `Real` or `Complex`; no extended-real ($\overline{\mathbb{R}}$) machinery is used. Some of the shortest paper witnesses (using $\ln 0 = -\infty$) cannot be expressed in our framework as a result; we use longer "clean-math" witnesses instead.

These are documented in each chunk's `notes`. None *contradict* the paper's universality claim, but each is a place where the formal artefact is strictly less ambitious than the paper prose.

11 Conclusion and acknowledgments

11.1 The 45/45 result

All 45 chunks of the EML formalisation are complete. The repository at `lambda_lab/proofs/eml/2603_21852/` holds:

- 45 chunk directories with bilingual prose, Lean targets, metadata, and verified results;
- a `lean_workspace/` tree that compiles cleanly under `lake env lean` on `leanprover/lean4:v4.28.0` + `Mathlib v4.28.0`;
- the eleven-conjunct umbrella theorem `main_completeness` (514 lines) collecting the constructive sub-cases;
- independent $\mathsf{EMLTerm}_{\mathbb{C}}$ proofs of the π and i witnesses;
- Mathematica search tools v1 and v2, with input specs for the deferred chunks;
- this report.

11.2 The paper's Lean aside

The paper's Supplementary §2.5 contains the line:

> *A formalization in Lean 4 would be a natural next step. However, the standard Lean 4 convention* $\log 0 = 0$ *(totality requirement) causes the shortest constant witnesses to fail without modification, so a formalization would require either adjusting the extended-value conventions or using longer witnesses that avoid* $\ln(0)$.

The present project chose the latter strategy: every witness in the umbrella theorem avoids $\ln(0)$ by either (a) keeping the argument positive by construction (the `set`/`show` chains in `c032_two`, etc.) or (b) restricting the existential to $x > 0$ where Mathlib's totality convention is irrelevant. No extended-real machinery, no `Classical.choice`, no `Nonempty` hacks.

11.3 Acknowledgments

- **Andrzej Odrzywołek** for the paper and the candid Supplementary Information, in particular for the explicit remark about the Lean formalisation difficulty which shaped our positivity strategy.
- **Aristotle, Harmonic AI** [2] for the project-archive submission interface; chunks like `006`–`018` closed without any human proof writing.
- **Wolfram Mathematica** for the `FullSimplify`-with-`TimeConstrained` infrastructure that `mma_eml_search_v2.wls` relies on.
- **The Mathlib community** [3] for the principal-branch `Complex.log` infrastructure that makes the π and i witnesses tractable.
- **Lean 4** [4] for the tactic framework and `lake env lean` verification harness.

References

[1] A. Odrzywołek. *All elementary functions from a single binary operator.* Preprint, 2026. `arXiv:2603.21852`.

[2] Harmonic AI. *Aristotle: project-archive theorem-proving for Lean 4.* `https://harmonic.fun`, accessed 2026.

[3] The Mathlib Community. *The Lean Mathematical Library.* `https://leanprover-community.github.io/mathlib4_docs/`, 2024. Version pinned in this project: `v4.28.0`.

[4] L. de Moura and S. Ullrich. The Lean 4 Theorem Prover and Programming Language. In *Proceedings of the 28th International Conference on Automated Deduction (CADE-28)*, 2021.

[5] Wolfram Research. *Mathematica, Version 14.* Champaign, IL, 2024.

[6] A. Macintyre and A. J. Wilkie. *On the decidability of the real exponential field.* In Kreiseliana: About and Around Georg Kreisel, ed. P. Odifreddi, 1996.

[7] D. Richardson. *Some undecidable problems involving elementary functions of a real variable.* J. Symbolic Logic 33(4), 1968.

A Appendix: representative Lean source

For reference, the full text of one self-contained chunk solution (`030_emlterm_for_zero.lean`) is reproduced below. This is a typical wave-3 deliverable: 25 lines, declares the inductive locally, and supplies a single-step `simp`-based proof of an existential.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

inductive EMLTerm : Type
  | one : EMLTerm
  | eml : EMLTerm → EMLTerm → EMLTerm
```

```
  deriving Repr

noncomputable def EMLTerm.eval : EMLTerm → ℝ
  | .one => 1
  | .eml t u => Real.exp (EMLTerm.eval t) - Real.log (EMLTerm.eval u)

theorem emlterm_for_zero : ∃ t : EMLTerm, EMLTerm.eval t = 0 := by
  -- Witness: eml one (eml (eml one one) one)
  -- eval = exp(1) - log(exp(exp(1) - log(1)))
  --      = exp(1) - log(exp(exp(1) - 0))
  --      = exp(1) - log(exp(exp(1)))
  --      = exp(1) - exp(1) = 0
  exact ⟨.eml .one (.eml (.eml .one .one) .one), by
    simp [EMLTerm.eval, Real.log_one, sub_zero,
          Real.log_exp, sub_self]⟩

end EML
```

B Appendix: a v2 spec file

Each Mathematica search invocation reads a JSON spec. The spec for chunk `034` is:

```
{
  "target": "Pi",
  "vars": [],
  "constraint": "True",
  "max_size": 31,
  "theorem_name": "emlterm_for_pi"
}
```

and for chunk `038`:

```
{
  "target": "x^2",
  "vars": ["x"],
  "constraint": "x > 0",
  "max_size": 13
}
```

The runtime then reads `target` as the Mathematica expression to match, `vars` as the list of free variables (each becomes a leaf alongside `1`), `constraint` as the symbolic side-condition fed to `Simplify`, and `max_size` as the iterative-deepening cap.

C Appendix: chunk metadata schema

The full `meta.json` schema (all keys observed across the 45 chunks):

- `id` (string) — canonical chunk ID, e.g. `006_eml_one_one_eq_e`.
- `title_pl, title_en` — bilingual titles.
- `paper_section, paper_quote` — pinpoint references back to `paper_extracted.md` or the Supplementary.
- `informal_pl, informal_en` — 2–3 sentence prose description of the claim.
- `kind` $\in$ {`definition`, `identity`, `theorem`, `calculator-equivalence`}.
- `difficulty` $\in \{1, \ldots, 5\}$.
- `lean_imports` (list) — Mathlib modules expected.
- `lean_target_signature` — the literal Lean theorem statement ending in `:= by sorry`.
- `depends_on` (list of chunk IDs) — forms the DAG.

- `status` ∈ {`pending`, `in_progress`, `partial`, `complete`, `failed`}.
- `aristotle_project_id` (UUID or `null`).
- `submitted_at, completed_at` (ISO 8601 UTC).
- `notes` — append-only decision log; the project's collective memory.
- `history` (list, optional) — prior submission attempts with their outcomes.
- `search_v2` (object, optional) — Mathematica search result block: `spec, result, max_size_reached, total_trees_generated, unique_signatures_deduped, numeric_matches, symbolic_checks, wall_clock_seconds, outcome, conclusion`.

D Appendix: reproducing the verification

A complete recipe for an independent verifier is:

1. Install `elan` and pin to `leanprover/lean4:v4.28.0` via `elan toolchain install`.
2. In `lambda_lab/proofs/eml/2603_21852/lean_workspace`, run `lake exe cache get` to fetch Mathlib v4.28.0.
3. Run `lake build` once to compile the workspace (this takes ∼10–15 minutes on a modern laptop, mostly Mathlib).
4. For each chunk solution file `EML/Solutions/<NNN>.lean`, run `lake env lean EML/Solutions/<NNN>.lean`. Exit code 0 with no `sorry` warning means the chunk verifies.
5. In particular, `lake env lean EML/Solutions/045_main_completeness_stub.lean` checks the umbrella theorem and the eleven inlined sub-proofs in one shot.

The Mathematica search tools require Wolfram Engine ≥ 13 and are invoked as `wolframscript -file lambda_lab/proofs/eml/tools/mma_eml_search_v2.wls <spec.json>`. Output is JSON on `stdout` and progress on `stderr`.

E Appendix: glossary of acronyms

EML
Exp-Minus-Log; the operator $\mathrm{eml}(x, y) = \exp(x) - \ln(y)$ introduced by Odrzywołek.

EDL
Exp-Divide-Log; the variant $\mathrm{edl}(x, y) = \exp(x)/\ln(y)$.

$-$ eml
negated EML, $-\mathrm{eml}(y, x) = \ln(x) - \exp(y)$.

EMLTerm
the inductive type with constructors `one` and `eml`; closed (no free variables).

$\mathsf{EMLTerm}_1$
one-variable extension with a `var` constructor.

$\mathsf{EMLTerm}_2$
two-variable extension with `varX, varY`.

$\mathsf{EMLTerm}_{\mathbb{C}}$
complex-valued extension; same constructors as EMLTerm, but `eval` returns $\mathbb{C}$ via `Complex.exp` and `Complex.log`.

Lg the macro $\mathrm{Lg}(t) := \mathrm{eml}(1, \exp(\mathrm{eml}(1, t)))$ from Supplementary §2.1; equals `Complex.log` on the principal branch.

Calc 0–3, Wolfram
the calculator-configuration rows of Table 2 of the main text.

K RPN code length of an EML witness, in nodes. K_{compiler} is the size produced by the paper's mechanical compiler; K_{direct} is the smallest size found by the direct exhaustive search. Both are tabulated for each primitive in Table 4 of the paper and Table S2 of the Supplementary.

CWE
`COMPLETE_WITH_ERRORS`, an Aristotle return status containing partial proofs and a diagnostic.